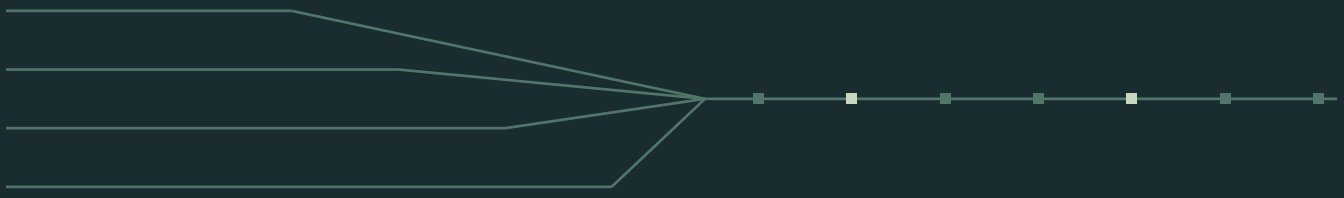
Автоматические признаки для рекомендаций

Yambda flat 50m · Deep Feature Synthesis · CatBoost



СВЯЗИ МЕЖДУ СУЩНОСТЯМИ → ФОРМУЛЫ → РАНЖИРОВАНИЕ

48
ручных признаков

462
признака DFS

30
дней в переборе

Автоматизировать выбор признаков.
Проверить качество. Разобраться со временем.

01 ЗАДАЧА

Автоматическое создание признаков для RecSys

Исследование на Yambda flat 50m

Постановка задачи

Для пары пользователь-трек нужно оценить релевантность по истории, доступной до момента запроса. Проверяю, можно ли автоматизировать построение признаков для CatBoostRanker и получить качество, близкое к расширенному ручному набору. Отдельный вопрос - как выбирать временные окна, не ограничиваясь несколькими привычными значениями.

Идея: задать сущности и допустимые операции, перечислить формулы DFS с окнами 1-30 дней и признаками свежести, затем сократить набор по обученной модели. В итоговом сравнении используются 48 ручных и 462 автоматических признака на одинаковых строках.

Основной результат

Модель	Validation NDCG@10	Повторный test NDCG@10
Ручные48	0.804589	0.808769
DFS462	0.804623	0.808187

Точечное качество близко. DFS не показал преимущества перед расширенным ручным набором, но позволил автоматически построить интерпретируемые признаки и указал на полезность недавней истории пользователя. Близость оценок сама по себе не доказывает эквивалентность методов.

Основную пару ручные48-DFS462 выбрали после открытия test: у ручного контроля исключили текущий канал взаимодействия, доступность которого в момент прогноза не обоснована. Поэтому повторный test здесь служит исследовательской проверкой, а не новым слепым подтверждением.

Что входит в отчёт

Данные и метки; диагностика времени; подход к признакам; схема эксперимента; метрики и интервалы; вычислительная стоимость; выводы и воспроизведение. Результаты прежнего некорректного cutoff используются только для объяснения найденной проблемы.

Источник: no_current_channel/comparison.csv; подробные пути к данным отчёта приведены в конце.

02 ДАННЫЕ

Данные и целевая переменная

Использована версия Yambda flat 50m, а не весь Yambda-5B [1, 2]. Исходный журнал содержит прослушивания и реакции: like, dislike, unlike, undislike. Идентификаторы пользователей, треков и исполнителей позволяют строить прямые и косвенные связи.

Характеристика	В выбранной версии
Все события	47 790 449
Прослушивания / реакции	46 467 212 / 1 323 237
Пользователи / треки	10 000 / 934 057
Полные дубликаты строк	236 230; сохранены

Как получены метки

Готовой метки релевантности для выбранной задачи в данных нет. Для каждого прослушивания реакции той же пары привязываются к ближайшему следующему концу прослушивания, если попадают в восстановленный интервал. Приоритет разметки: **like - 2**; иначе **dislike или доля прослушивания меньше 35% - 0**; иначе **1**. Если привязаны обе реакции, приоритет у like. Unlike/undislike напрямую метку не задают.



01 / Распределение доли прослушивания

Распределение сосредоточено около ранних пропусков и конца трека. Порог 35% - ручная эвристика, а не выведенная из этого графика оптимальная граница. Значения выше 100% встречаются в исходном журнале. Сравнение относится к этой разметке поведения, а не к независимо измеренной полезности рекомендаций.

Источник: docs/results/dataset-analysis/{snapshot.json,ratio_counts.csv}; task_1/src/c4_targets.py.

03 ДИАГНОСТИКА

Почему пришлось изменить время запроса

В ранней версии признаки рассчитывались к восстановленному началу прослушивания. Для его вычисления использовалась **текущая** доля прослушивания R. Таким образом, момент расчёта уже зависел от будущего исхода целевого события.

Механика канала

Сохранённая формула: **S = E - 5 - ceil(L * R / 100)**, где E - исходный timestamp прослушивания, L - длина трека в секундах. Она совпала с восстановленным временем на всех 2 039 017 train- и 2 882 983 validation-строках прежнего сравнения.

Исходные времена и длины лежат на пятисекундной сетке. Поэтому остаток пользовательской свежести F по модулю 5 зависел от текущего R: **F mod 5 = (-ceil(L * R / 100)) mod 5**. Даже если R не подать в модель напрямую, он может попасть в неё через временной признак.

Эксперименты диагностики

Проверка на прежней validation	Наблюдение
Таблица частот по фазе F mod 5, построенная только на train	ROC AUC для label=0 против остальных: 0.7542
Перестановка свежести внутри (L, floor(F/5)); ручная модель заморожена	NDCG: 0.849833 → 0.806796
Та же перестановка; DFS-модель заморожена	NDCG: 0.847202 → 0.802563

Перестановка не использовала метки, меняла свежесть максимум на 4 секунды и оставляла прочие входы фиксированными. Это показывает чувствительность обученных моделей к найденному каналу. Падение не является оценкой причинной доли утечки и не равно качеству после переобучения.

Что изменено

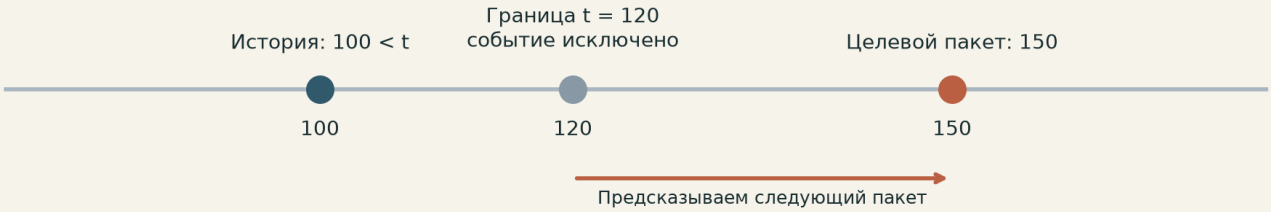
Момент запроса заменён предыдущим исходным timestamp пользователя. После этого заново построены целевые строки, признаки, split и группы и переобучены модели. Восстановленное начало больше не задаёт cutoff признаков; оно сохранено в привязке реакций. Старые высокие метрики не используются как основной результат исследования.

Источник: task_1/results/analysis/start_ts_{formula,phase,phase_permutation,dfs_phase_permutation}_20261024_v1; docs/start-time-audit.md.

04 ВРЕМЯ

Подход: прогноз следующего пакета

Прослушивания пользователя с одинаковым исходным timestamp объединяются в пакет. Для целевого пакета момент запроса **t** равен предыдущему **строго меньшему** timestamp того же пользователя. Первый пакет исключается. История содержит только исходные события с timestamp < t; пакет прямо на границе t тоже исключён. Временное окно: **[t-w, t)**.



02 / Момент запроса и доступная история

Для равных исторических timestamp порядок задаёт исходный ID строки, без сортировки по метке или доле прослушивания. Внутренний порядок событий в пятисекундном бине неизвестен. Интерпретация исходного timestamp как времени доступности результата остаётся допущением: карточка данных однозначно не описывает момент отправки и доступности логов.

Часть	Cutoff, дни с начала	Строки	Группы
Train	[225, 270)	2 021 867	40 000
Validation	[270, 285)	2 879 535	57 097
Повторный test	От 285 до конца	3 160 068	61 964

В train и validation исходный timestamp целевого пакета должен быть меньше правой границы минус GAP 1800 секунд. У test конечного GAP нет. Из train выбраны 40 000 целых смешанных групп с seed 42; validation и test взяты полностью в рамках правил отбора.

Если прикреплённый к цели like/dislike уже произошёл раньше t, строка исключается: 448 в train, 170 в validation и 191 в test. Это правило зависит от исхода и меняет популяцию оценки.

История начинается с начала журнала и обновляется внутри validation/test по мере доступности событий. Веса модели фиксированы. Текущий label и доля прослушивания не входят в признаки. Длина целевого трека считается заранее известным каталоговым атрибутом; текущий is_organic исключён.

Источник: task_1/src/c4_targets.py; docs/c4.md; docs/c4-test.md; no_current_channel/result.json.

05 ПРИЗНАКИ

Подход: ручные и автоматические признаки

Ручные48

Ручной набор содержит длину трека и 47 исторических признаков. К исходным item-, user-item- и user-artist-признакам после анализа DFS добавлены 7 признаков недавнего поведения пользователя. Таким образом, это усиленный контроль, сформированный с учётом результатов автоматического поиска.

История	Примеры и окна
Трек	Число прослушиваний, skip-rate, like/listen, средняя доля прослушивания; 1/7/30/90 дней
Пользователь-трек	Число прослушиваний и пропусков; 7/14/30/90 дней; затухание 1/7/30 дней; давность и состояние реакций
Пользователь-исполнитель	Счётчики, пропуски и затухание по исполнителю
Пользователь	Доля прослушивания и skip последнего события; skip-rate последних 5/20 событий и 24 часов; число событий за последние 24 часа; давность

DFS462

Deep Feature Synthesis [3] строит формулы через связи между сущностями. Здесь используются **Users, Items, UserItem, UserArtist** и журнал Interactions. Сначала агрегируем доступную историю по ключу сущности, затем присоединяем результат к целевой строке. Так средняя историческая метка пары user-item становится признаком следующего взаимодействия.

Featuretools [4] создаёт определения формул. Собственный кумулятивный исполнитель считает поддерживаемую грамматику, переиспользуя состояние между запросами. Это ускорение выбранного подмножества формул, а не универсальная реализация всех возможностей библиотеки.

Числовые поля истории DFS: **label, track_length_seconds, is_organic**. Операции: count, mean, std, min, max, допустимые произведения и давность первого/последнего события. Историческая непрерывная доля прослушивания напрямую в DFS не включена, хотя ручной набор её использует. Исходные представления сравниваемых методов поэтому различаются.

Источник: task_1/src/rolling_manual_features.py; cumulative_features.py; docs/feature-analysis.md; no_current_channel/protocol.json.

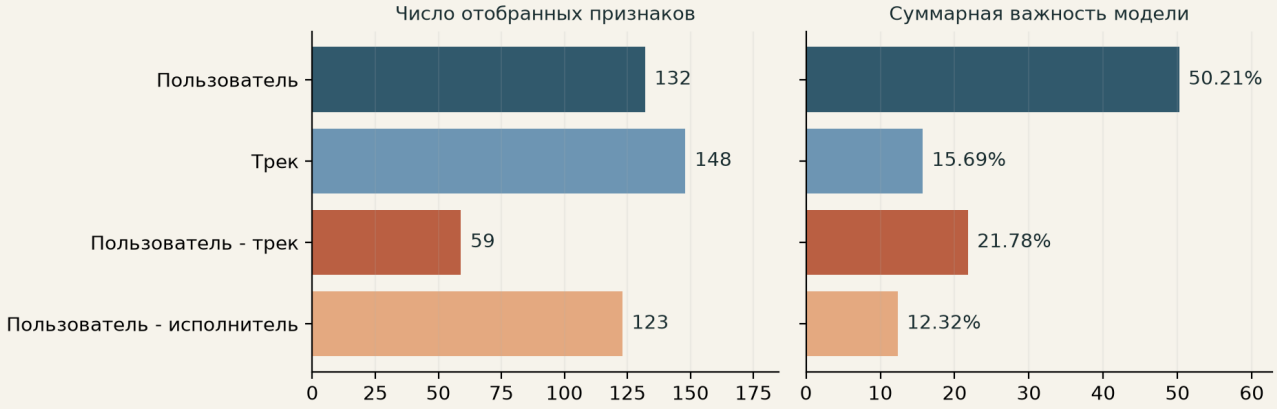
06 ПОИСК

Эксперименты: поиск и отбор формул

Для каждой из четырёх сущностей перечислены 103 базовые формулы в 31 режиме: вся история и окна от 1 до 30 дней. Добавлены два признака давности для каждой сущности и два текущих контекстных поля. Всего **4 × (103 × 31 + 2) + 2 = 12 782 кандидата**. Произведения агрегатов строятся внутри одного окна; сочетания разных окон не перебираются.

Этап	Что сделано
Подвыборка	9 999 train-строк / 183 группы; 5 000 validation-строк / 96 групп
Проверка top-k	Важность LossFunctionChange на train; размер набора сравнивался по validation NDCG
Сокращение	Top-k не улучшил полный набор. Оставлены все 462 признака, использованные в разбиениях 91-деревной модели
Итоговый протокол	Фиксированный список перенесён на исправленный cutoff без повторного отбора или тюнинга

Отбор первоначально выполнялся при прежнем восстановленном cutoff. Поэтому 462 - фиксированный унаследованный набор, а не оптимальное число признаков для исправленного времени. Текущие контекстные поля в отобранные 462 не вошли; исторический is_organic сохранился.



03 / Состав 462 признаков и важности по сущностям

В набор вошли 132 пользовательских, 148 item-, 59 user-item- и 123 user-artist-признака. Число признаков не определяет их важность: пользовательская ветвь получила около половины суммарной PredictionValuesChange итогового DFS-ранкера.

Автоматизация перечисляет формулы и окна внутри заданной грамматики. Сущности, исходные поля, операции, временной протокол и вычислительный бюджет по-прежнему задаются вручную.

Источник: docs/results/feature-analysis/entity_summary.csv; docs/feature-analysis.md; temporal_topk_20261023_v2 и temporal_used_20261023_v1.

07 ИНТЕРПРЕТАЦИЯ

Какие признаки выбрала модель



04 / Важности формул с разной длиной окна

51 признак использует всю доступную историю, 411 - конечное окно. В отобранном наборе представлены все длины от 1 до 30 дней. Самый заметный оконный блок - один день (18.92% суммы важностей); вся история даёт 38.55%, включая признаки давности.

Примеры с понятным смыслом

Признак	Интерпретация	Важность
Давность последнего события пользователя	Насколько недавно пользователь слушал музыку	14.21%
max(label) × mean(label), пользователь, 1 день	Сочетание лучшей и средней исторической реакции за сутки	7.03%
mean(label), пользователь-трек, вся история	Средняя оценка предыдущих прослушиваний этой пары	6.71%
mean(label), пользователь, 3 дня	Недавний общий характер реакций пользователя	3.64%

Средняя метка user-item считается по предыдущим доступным событиям пары, без текущей цели. Для неё полезны повторные прослушивания одного трека. Пользовательская свежесть и недавние пропуски описывают общий контекст поведения; именно этот блок затем усилил ручной контроль.

Важности - описание обученной модели [6]. Окна и агрегаты коррелируют, поэтому сумма важностей не измеряет причинный вклад и не доказывает оптимальность одного дня. Для такого вывода понадобилось бы отдельное сравнение с удалением/заменой блоков и новым закрытым периодом.

Источник: docs/results/feature-analysis/{window_summary.csv,top_features.csv,prediction_values_change.csv}.

08 ЭКСПЕРИМЕНТ

Эксперименты: модель и оценка

Общий протокол обучения

Обе модели - CatBoostRanker с YetiRank [5]. Целевые строки, группы, метки и временные границы совпадают. Границы квантования обучаются на train. Для ручных48 физическая матрица содержит 49 колонок, но текущий is_organic исключён через ignored_features=[0].

Параметр	Общее значение
Глубина / learning_rate	6 / 0.1
Максимум деревьев / early stopping	1600 / 80
Seed / CPU-потоки	42 / 4
Bootstrap / subsample / rsm	MVS / 0.8 / 1.0
border_count / random_strength	254 / 1.0
l2_leaf_reg	1e-20; унаследованная конфигурация

Параметры перенесены из предыдущих сравнений. После исправления cutoff новый поиск гиперпараметров не проводился. Это ограниченная проверенная конфигурация, а не гарантированно лучший бустинг для каждой системы признаков.

Метрики и кандидаты

NDCG@10 считается с линейной релевантностью 0/1/2: DCG@10 = sum(label_i / log2(i+1)); NDCG = DCG / IDCG. Итог - среднее по группам с одинаковым весом. Точные ничьи score разрешаются общей фиксированной перестановкой seed 42. Встроенная early-stopping-метрика CatBoost разрешает их иначе.

Дополнительная метрика - **QueryAUC:type=Ranking** по исходным score. Она не подменяется бинарным ROC AUC. Основное сравнение относится к актуальным ранкерам.

Группа - пользователь и день cutoff: uid*1000 + floor(t/DAY). Удалены группы с постоянной меткой. Кандидаты - фактически наблюдавшиеся прослушивания, без negative sampling. Они относятся к разным моментам запроса одного дня. Это оценка ранжирования наблюдавшихся событий, а не выдачи всего каталога в один момент.

Источник: no_current_channel/protocol.json; task_1/src/train_dfs_catboost.py; c4_targets.py; документация CatBoost [5].

09 РЕЗУЛЬТАТЫ

Результаты: ручные48 и DFS462

Ручные48

0.808769

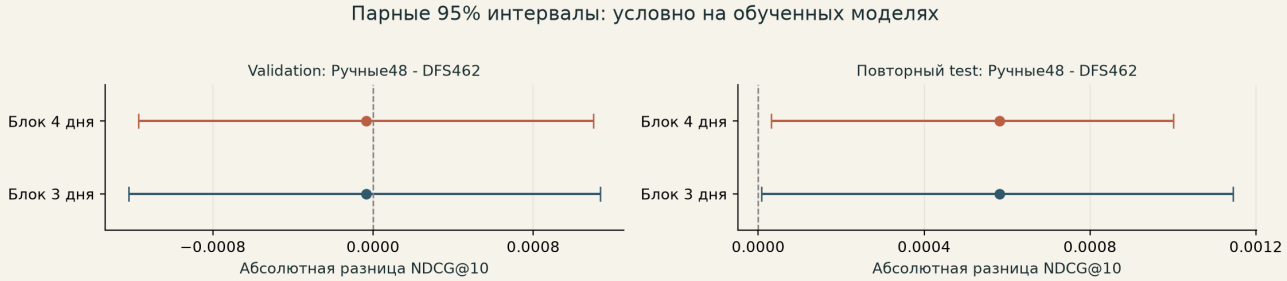
NDCG@10 / повторный test

DFS462

0.808187

NDCG@10 / повторный test

Часть / модель	NDCG@10	QueryAUC
Validation: ручные48	0.804589	0.677678
Validation: DFS462	0.804623	0.676805
Повторный test: ручные48	0.808769	0.677053
Повторный test: DFS462	0.808187	0.676752



05 / Парные интервалы разницы NDCG@10

Разница определена как **ручные48 - DFS462**: на validation -0.00003418, на повторном test +0.00058196 NDCG. Для парного циклического блочного bootstrap по дням использованы 20 000 реплик, блоки 3 и 4 дня, seed 42 и двусторонние 95% percentile-интервалы.

Часть	Блок	Нижняя граница	Верхняя граница
Val	3 дня	-0.00121999	0.00113397
Val	4 дня	-0.00117092	0.00110052
Повторный test	3 дня	0.00000761	0.00114435
Повторный test	4 дня	0.00003183	0.00100143

Validation-интервалы включают ноль. На test они положительны, но нижние границы практически нулевые. Это не основание заявлять устойчивое превосходство ручного метода или эквивалентность систем: пара выбрана после открытия test, seed обучения один, а интервалы условны на уже обученных моделях. Поправка на множественные сравнения не применялась.

Источник: task_1/results/c4/no_current_channel/{comparison.csv,bootstrap_summary.csv}.

10 СТОИМОСТЬ

Вычислительная стоимость

Наблюдавшееся время

Построение матрицы	Train	Validation
Ручная матрица: 49 колонок	47.97 с	64.43 с
DFS: 462 колонки	188.10 с	234.24 с

Обучение	Fit, минуты	Деревья
Ручные48	17.84	911
DFS462	26.73	1043

Ручные48 переиспользуют матрицу49 с игнорируемой колонкой. Их fit выполнялся с пониженным приоритетом параллельно аудиту, поэтому эти времена не являются чистым сравнением скорости. Генерация DFS в наблюдавшемся прогоне дороже. Число колонок само по себе не измеряет потребление всей памяти WSL.

Этап train/validation трёх первоначальных исправленных моделей, включая построение признаков, квантование и сохранение, занял 67.04 минуты после подготовки истории. Пиковый anonymous RSS процессов этого этапа составил 5.74 GiB; это не вся память WSL. Test-этап занял 7.47 минуты. Дополнительный fit ручных48 учитывается отдельно.

Это подтверждает выполнимость выбранного ограниченного набора на миллионах целевых строк. Стоимость всех 12 782 кандидатов на полном объёме не измерена: широкий поиск выполнялся на малой подвыборке. Перенос на миллиарды событий этими замерами не проверен.

Источник: generation_{manual,dfs}_{train,val}.json; c4_packet_20261024_v1/result.json; c4_test_20261024_v1/result.json; no_current_channel/comparison.csv.

Воспроизведение эксперимента

Репозиторий: github.com/GodNaoborot/T-labs-recsys. Главные инструкции - docs/reproduction.md, итоговый расчёт - final-comparison.ipynb. Быстрый режим ноутбуков читает опубликованные таблицы и строит графики без моделей и нового обучения.

Установка и запуск отчётов

Python 3.12, Linux или WSL/Ubuntu. Команды выполняются из корня клонированного проекта. Создаётся отдельное окружение; выполненные копии ноутбуков сохраняются в отдельную папку.

Шаг	Команда
Окружение	python3.12 -m venv task_1/.venv-featuretools
Активация	source task_1/.venv-featuretools/bin/activate
Зависимости	python -m pip install -r task_1/requirements.txt
Проверка установки	python -m pip check
Отчёты без fit	python task_1/src/run_report_notebooks.py --output-dir task_1/.validation/reports

Повтор из исходных данных

Закреплённая версия данных скачивается по манифесту. Новый run-name сохраняет расчёт отдельно от опубликованных результатов. Следующая команда запускает обучение; --include-test означает повторную оценку уже открытого периода.

```
python task_1/src/run_all_experiments.py --run-name repeat_main
  --experiments channel-control --download --include-test --execute
```

В терминале это одна команда. Граф готовит историю, целевые строки и матрицы, обучает три первоначальных модели исправленного протокола и контроль ручные48. Исторический поиск и Optuna в эту ветвь не входят. Для нового расчёта нужна отдельная чистая копия; ориентир ресурсов инструкции - 12 GiB RAM и не менее 40 GB свободного диска.

Что проверено фактически

В отдельной чистой копии повторены исходные три модели исправленного протокола: шесть матриц побайтно совпали, validation/test-предсказания и метрики воспроизвелись. Для ручных48 опубликованы исходные fit/evaluate-скрипты, параметры и хеши; свежие test-предсказания совпали точно. Новый переносимый CLI проверен на синтетическом цикле, но его полный повтор на реальных данных не выполнялся. Все исторические обучения и Optuna заново не повторялись.

Источник: docs/reproduction.md; docs/results/reproduction/main_verification.json; task_1/reproduction/channel_control_{fit,evaluate}.py.

Выводы и границы исследования

Что удалось проверить

1. Автоматическое построение формул работает на выбранном масштабе. Перечислены 12 782 кандидата; фиксированный набор из 462 признаков посчитан к миллионам целевых строк с переиспользованием исторических статистик.

2. Качество близко к расширенному ручному контролю. На validation оценки практически совпали; на повторном test ручные48 выше на 0.000582 NDCG. Доказательства превосходства DFS или формальной эквивалентности нет.

3. DFS полезен как инструмент поиска. Он выявил заметные пользовательские признаки свежести и недавних реакций. Их смысл можно объяснить, а несколько простых признаков позволили усилить ручной набор. Это наблюдение о ходе исследования, не причинная абляция каждого найденного признака.

4. Проверка времени оказалась существеннее прироста метрики. Восстановленный cutoff зависел от текущего исхода; найденный канал исключён из нового момента расчёта. Предыдущие высокие оценки не считаются валидным подтверждением качества.

Что ограничивает вывод

Один датасет, один актуальный временной split и seed обучения; повторно использованные validation/test; подбор основной пары после открытия test. Старые дополнительные окна не являются повторением финального исправленного эксперимента.

Метки получены эвристически; восстановленное время остаётся в привязке реакций. Доступность результатов в исходный timestamp и каталоговой длины - допущения. Отбор смешанных user-day групп и исключение ранних реакций зависят от исходов. Оценка не распространяется автоматически на полный каталог и реальные выдачи.

Наборы используют разные числовые представления истории. Формулы и гиперпараметры перенесены из прежних этапов; новый оптимальный набор не искался. Не проверялись миллиарды событий, другие датасеты или широкий набор обучающих seed.

Что проверять дальше

Зафиксировать текущие два метода и оценить их на новом закрытом периоде с несколькими seed; согласовать семантику времени с владельцами данных; отдельно проверить непрерывную долю прослушивания в грамматике DFS. Эти проверки предложены как продолжение и не включены в полученные результаты.

Дальше можно совместить LLM с DFS: по схеме сущностей и описанию полей LLM предлагает новые связи и формулы, а существующий конвейер проверяет и рассчитывает поддерживаемые признаки. Кандидаты проходят проверку доступности во времени. Сравнить DFS и LLM+DFS стоит при одинаковом бюджете на новом закрытом периоде. Этот вариант пока не реализован и не оценивался.

Источники и проверяемые артефакты

[1] Ploshkin A. et al. Yambda-5B: A Large-Scale Multi-modal Dataset for Ranking And Retrieval. 2025, v2. Описание исходного датасета и его полей; здесь выбран собственный протокол оценки.

Открыть первоисточник

[2] Yandex. Yambda dataset card; закреплённая ревизия dd6f3a19eef5866e346c3270e098baa641a44948. Схема, варианты данных и лицензия Apache-2.0. Исходный Yambda не включается в кодовую MIT-лицензию проекта.

Открыть первоисточник

[3] Kanter J. M., Veeramachaneni K. Deep Feature Synthesis: Towards Automating Data Science Endeavors. DSAA, 2015. Реляционное построение признаков через связи и композиции операций.

Открыть первоисточник

[4] Featuretools. Deep Feature Synthesis API. Определения DFS-формул и параметры генерации.

Открыть первоисточник

[5] CatBoost. Ranking: objectives and metrics. YetiRank, NDCG и QueryAUC Ranking.

Открыть первоисточник

[6] CatBoost. Feature importance. Интерпретация PredictionValuesChange и LossFunctionChange.

Открыть первоисточник

Откуда взяты числа и рисунки

Все пути ниже относительны корню репозитория. Сборщик читает небольшие JSON/CSV, а не веса моделей или матрицы взаимодействий.

Главная таблица и интервалы: task_1/results/c4/no_current_channel/

Диагностика времени: task_1/results/analysis/; docs/start-time-audit.md

Выбранные формулы и важности: docs/results/feature-analysis/

Агрегаты анализа данных: docs/results/dataset-analysis/

Полный повтор и область проверки: docs/results/reproduction/main_verification.json

Исходные скрипты контроля: task_1/reproduction/channel_control_fit.py; channel_control_evaluate.py

Сборка этого отчёта: task_1/reproduction/build_editorial_report.py